Varendra University

Dept. of Computer Science & Engineering (CSE)

Flow Control Instructions (Chapter-6)

Course Title: **Microprocessor and Assembly Language**
Course No: **CSE 3205**

Prepared By:
Md. Adnan Sami
Lecturer, Dept. of CSE, VU

Outline

1. An example of Jump.
2. The CMP instruction.
3. Example 6.1.
4. The JMP Instruction.
5. High-Level Language Structure.

VARENDRA UNIVERSITY
বরেন্দ্র বিশ্ববিদ্যালয়

An example of Jump

Program Listing PGM6_1.ASM

```
1:  TITLE   PGM6_1: IBM CHARACTER DISPLAY
2:  .MODEL          SMALL
3:  .STACK          100H
4:  .CODE
5:  MAIN            PROC
6:          MOV     AH,2        ;display char function
7:          MOV     CX,256      ;no. of chars to display
8:          MOV     DL,0        ;DL has ASCII code of null cha
9:  PRINT_LOOP:
10:         INT     21h         ;display a char
11:         INC     DL          ;increment ASCII code
12:         DEC     CX          ;decrement counter
13:         JNZ     PRINT_LOOP  ;keep going if CX not 0
14: ;DOS exit
15:         MOV     AH,4CH
16:         INT     21h
17: MAIN    ENDP
18:         END     MAIN
```

JNZ = Jump if Not Zero

The CMP instruction

This instruction compares destination and source by computing **destination contents minus source contents**. The result is not stored, but the flags are affected.

```
CMP  destination, source
```

For Example:

```
CMP   AX, BX
```

where AX = 7FFFh, and BX = 0001. The result of CMP AX,BX is 7FFFh 0001h = 7FFEh.

Example 6.1

Assembly Code Problem: Suppose AX and BX contain signed numbers. Write some code to put the biggest one in CX.

Solution:

```
        MOV   CX,AX          ;put AX in CX
        CMP   BX,CX          ;is BX bigger?
        JLE   NEXT           ;no, go on
        MOV   CX,BX          ;yes, put BX in CX
NEXT:
```

JLE = Jump if Less than or Equal

Example 6.1

Full Code:

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
 MAIN PROC
        MOV AX,6H
        MOV BX,10H
        MOV CX,AX
        CMP BX,CX
        JLE NEXT
        MOV CX,BX
        NEXT:
 MOV AH,4CH
 INT 21H
 MAIN ENDP
 END MAIN
```

VARENDRA UNIVERSITY
বরেন্দ্র বিশ্ববিদ্যালয়

The JMP Instruction

The **JMP** *(jump)* instruction causes an unconditional transfer of control **(unconditional jump)**. The syntax is

```
JMP  destination
```

where destination is usually a label in the same segment as the JMP itself (see Appendix F for a more general description).

JMP can be used to get around the range restriction of a conditional jump. For example, suppose we want to implement the following loop:

```
TOP:

;body of the loop
DEC  CX                ;decrement counter
JNZ  TOP               ;keep looping if CX > 0
MOV  AX,BX
```

and the loop body contains so many instructions that label TOP is out of range for JNZ (more than 126 bytes before JMP TOP). We can do this:

```
TOP:

;body of the loop

                DEC  CX         ;decrement counter
                JNZ  BOTTOM     ;keep looping if CX > 0
                JMP  EXIT
BOTTOM:
                JMP  TOP
EXIT:
                MOV  AX,BX
```

The JMP Instruction

Table 6.1 Conditional Jumps

Signed Jumps

Symbol	*Description*	*Condition for Jumps*
JG/JNLE	jump if greater than jump if not less than or equal to	ZF = 0 and SF = OF
JGE/JNL	jump if greater than or equal to jump if not less than or equal to	SF = OF
JL/JNGE	jump if less than jump if not greater than or equal	SF <> OF
JLE/JNG	jump if less than or equal jump if not greater than	ZF = 1 or SF <> OF

The JMP Instruction

Unsigned Conditional Jumps

Symbol	*Description*	*Condition for Jumps*
JA/JNBE	jump if above	CF = 0 and ZF = 0
	jump if not below or equal	
JAE/JNB	jump if above or equal	CF = 0
	jump if not below	
JB/JNAE	jump if below	CF = 1
	jump if not above or equal	
JBE/JNA	jump if equal	CF = 1 or ZF = 1
	jump if not above	

The JMP Instruction

Single-Flag Jumps

Symbol	*Description*	*Condition for Jumps*
JE/JZ	jump if equal	ZF = 1
	jump if equal to zero	
JNE/JNZ	jump if not equal	ZF = 0
	jump if not zero	
JC	jump if carry	CF = 1
JNC	jump if no carry	CF = 0
JO	jump if overflow	OF = 1
JNO	jump if no overflow	OF = 0
JS	jump if sign negative	SF = 1
JNS	jump if nonnegative sign	SF = 0
JP/JPE	jump if parity even	PF = 1
JNP/JPO	jump if parity odd	PF = 0

High-Level Language Structure

Branching Structure:

- IF-THEN
- IF-THEN-ELSE
- CASE

Branches with Compound Conditions:

- AND Conditions
- OR Conditions

Looping Structure:

- For Loop
- While Loop
- Repeat Loop

VARENDRA UNIVERSITY
বরেন্দ্র বিশ্ববিদ্যালয়

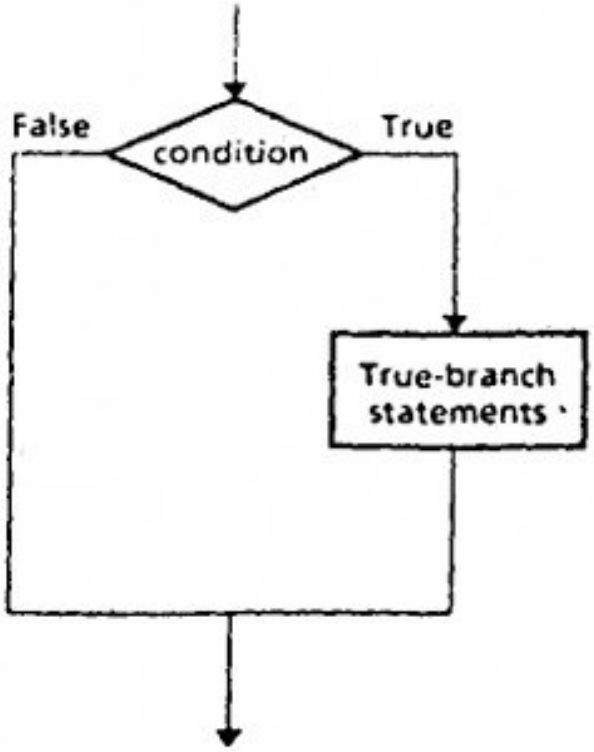
High-Level Language Structure

Branching Structure: *(IF-THEN)*

Syntax:

```
IF condition is true,
 THEN
  execute true-branch statements
END_IF
```

The condition is an expression that is true or false. If it is true, the true-branch statements are executed. If it is false, nothing is done and the program goes on to whatever follows.



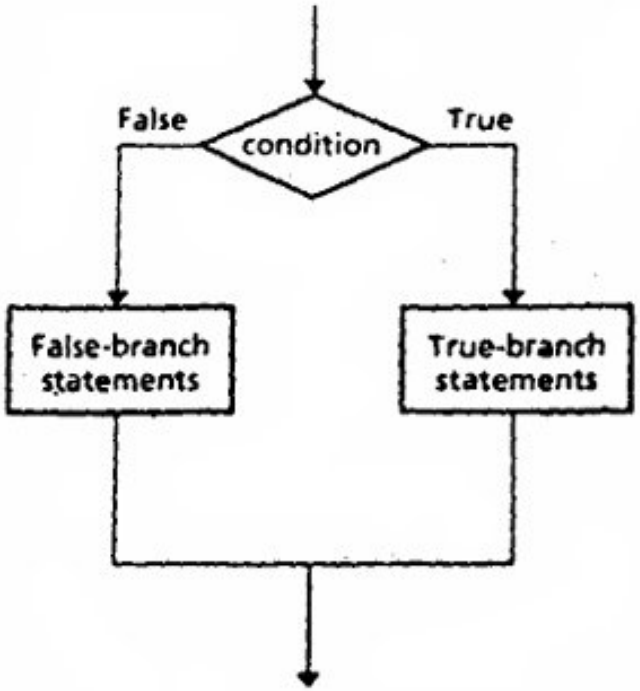
High-Level Language Structure

Branching Structure: *(IF-THEN-ELSE)*

Syntax:

```
IF condition is true
 THEN
  execute true-branch statements
 ELSE
  execute false-branch statements
END_IF
```

if condition is true, the true-branch statements are executed. If condition is false, the false-branch statements are done.



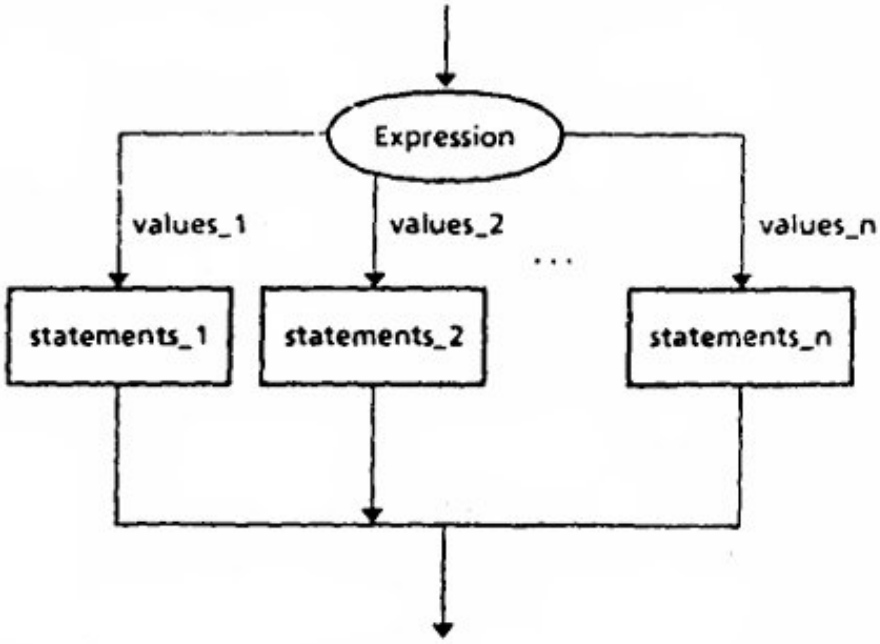
High-Level Language Structure

Branching Structure: *(CASE)*

Syntax:

```
CASE expression
    values_1:  statements_1
    values_2:  statements_2
    .
    .
    .
    values_n:  statements_n
END_CASE
```

In this structure, expression is tested; if its value is a member of the set values_i, then statements_i are executed.



Example 6.5

Assembly Code Problem: If AL contains 1 or 3, display "o"; if AL contains 2 or 4, display "e".

- **Done in Lab.**
- **Follow this code.**

Example 6.5

Solution:

```
CASE AL
   1,3: display 'o'
   2,4: display 'e'
END_CASE
```

The code is

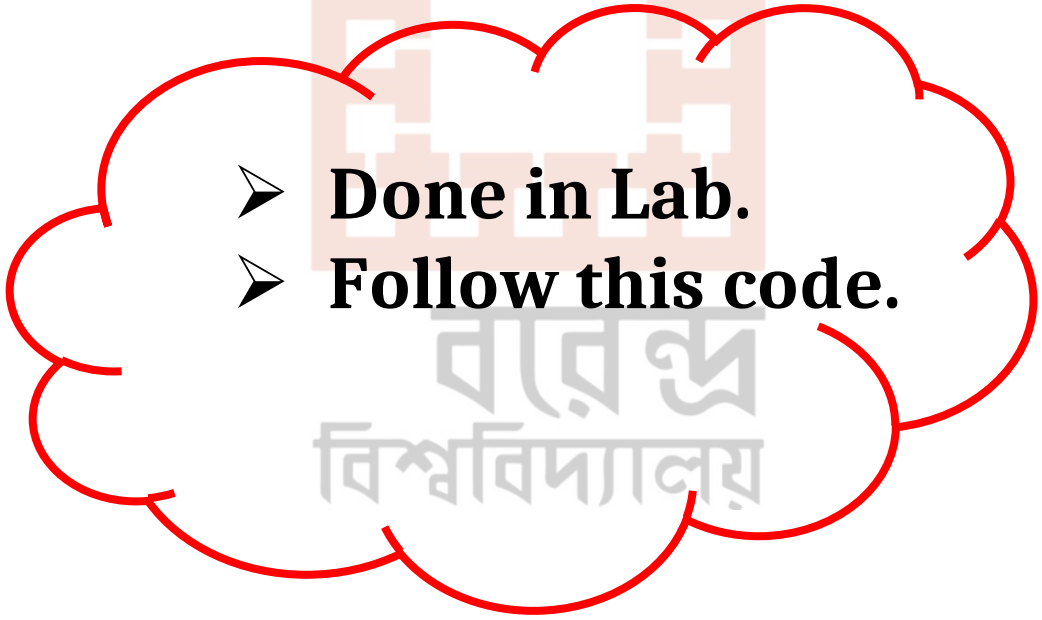
JE = Jump if Equal

```
;case AL
; 1,3:
                CMP   AL,1        ;AL = 1?
                JE    ODD         ;yes, display 'o'
                CMP   AL,3        ;AL = '3'
                JE    ODD         ;yes, display 'o'
; 2,4:
                CMP   AL,2        ;AL = 2?
                JE    EVEN        ;yes, display 'e'
                CMP   AL,4        ;AL = 4?
                JE    EVEN        ;yes, display 'e'
                JMP   END_CASE    ;not 1..4
ODD:                              ;display 'o'
                MOV   DL,'o'      ;get 'o'
                JMP   DISPLAY     ;go to display
EVEN:                             ;display 'e'
                MOV   DL,'e'      ;get 'e'
DISPLAY:
```

Example 6.5

Assembly Code Problem: If AL contains 1 or 3, display "o"; if AL contains 2 or 4, display "e".

Full Code:



High-Level Language Structure

Branches with Compound Conditions: *(AND Conditions)*

Example 6.6

Assembly Code Problem: Read a character, and if it's an uppercase letter, display it.

Solution:

```
Read a character (into AL)
IF ('A' <= character) and (character <= 'Z')
 THEN
  display character
END_IF
```

High-Level Language Structure

Branches with Compound Conditions: *(AND Conditions)*

```
;read a character
                MOV   AH,1          ;prepare to read
                INT   21H           ;char in AL
;if ('A' <= char) and (char <= 'Z')
                CMP   AL,'A'        ;char >= 'A'?
                JNGE  END_IF        ;no, exit
                CMP   AL,'Z'        ;char <= 'Z'?
                JNLE  END_IF        ;no, exit
;then display char
                MOV   DL,AL         ;get char
                MOV   AH,2          ;prepare to display
                INT   21H           ;display char
END_IF:
```

High-Level Language Structure

Branches with Compound Conditions: *(AND Conditions)*

Example 6.6

Assembly Code Problem: Read a character, and if it's an uppercase letter, display it.

- **Done in Lab.**
- **Follow this code.**

VARENDRA UNIVERSITY
বরেন্দ্র বিশ্ববিদ্যালয়

High-Level Language Structure

Branches with Compound Conditions: *(OR Conditions)*

Assembly Code Problem: Read a character. If it's y or Y, then print it *VU CSE*.

- **Done in Lab.**
- **Follow this code.**

VARENDRA UNIVERSITY
বরেন্দ্র বিশ্ববিদ্যালয়

High-Level Language Structure

Looping Structure: *(For Loop)*

A loop is a sequence of instructions that is repeated.

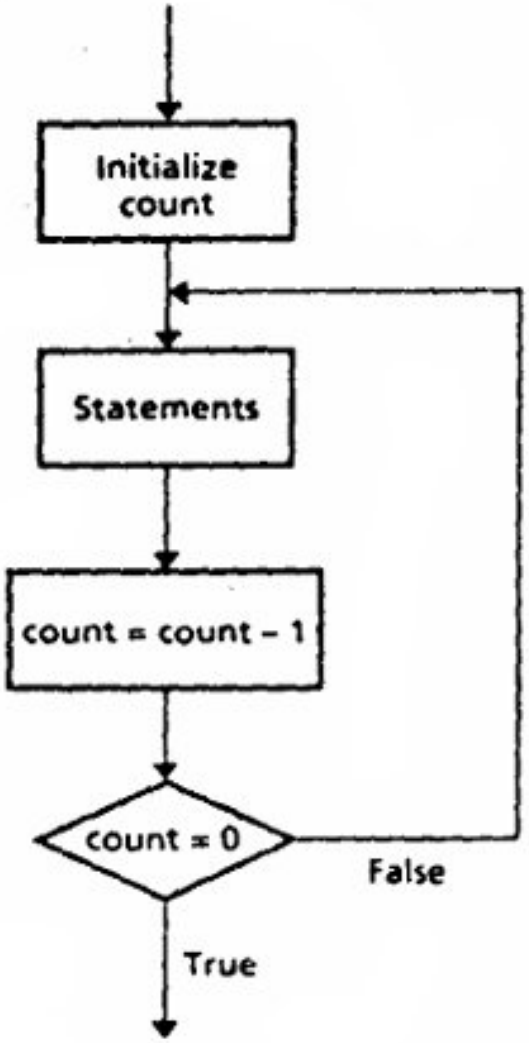
Syntax:

```
                ;initialize CX to loop_count
TOP:
                ;body of the loop
                LOOP TOP
```

VARENDRA UNIVERSITY
বরেন্দ্র বিশ্ববিদ্যালয়

High-Level Language Structure

Looping Structure: *(For Loop)*



High-Level Language Structure

Looping Structure: *(For Loop)*

Assembly Code Problem: Write a count-controlled loop to display a row of 80 stars. *(Example 6.8)*

The code is

```
                MOV   CX,80      ;number of stars to display
                MOV   AH,2       ;display character function
                MOV   DL,'*'     ;character to display
TOP:
                INT   21h        ;display a star
                LOOP  TOP        ;repeat 80 times
```

High-Level Language Structure

Looping Structure: *(For Loop)*

Assembly Code Problem: Write a count-controlled loop to display a row of 80 stars.

Full Code:

- **Done in Lab.**
- **Follow this code.**

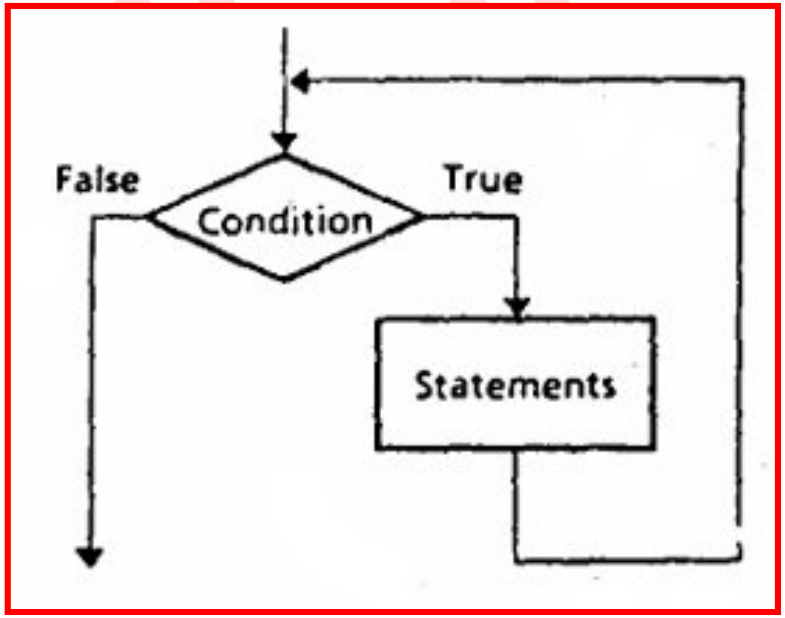
High-Level Language Structure

Looping Structure: *(While Loop)*

Syntax:

This loop depends on a condition. In pseudocode,

```
WHILE condition DO
  statements
END_WHILE
```



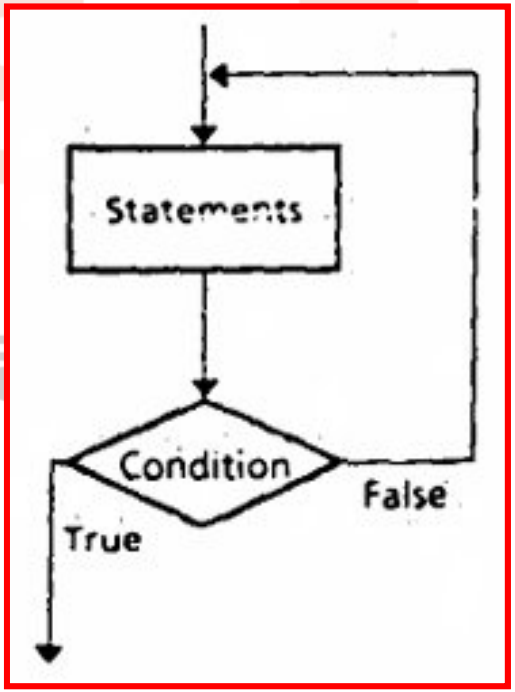
High-Level Language Structure

Looping Structure: *(Repeat Loop)*

Syntax:

Another conditional loop is the REPEAT LOOP. In pseudocode,

```
REPEAT
statements
UNTIL condition
```



High-Level Language Structure

While Versus Repeat Loop

- WHILE loop can be bypassed if the terminating condition is initially false, whereas the statements in a REPEAT loop must be done at least once.

- REPEAT loop has only a conditional jump at the end. But WHILE loop has two jumps: a conditional jump at the top and a JMP at the bottom.

VARENDRA UNIVERSITY
বরেন্দ্র বিশ্ববিদ্যালয়